

¿Qué vimos en el curso de "Lenguaje en C"?

1. Variables en lenguaje C:

- Se han enunciado los tipos de datos básicos y calificadores en lenguaje C.
- Se ha enunciado el concepto de variable en lenguaje C.
- Se ha indicado la sintaxis a la hora de usar identificadores de variables.

2. Operadores aritméticos en lenguaje C:

- Se han enunciado los operadores aritméticos en C.
- Se ha indicado cómo se evalúan estos operadores.
- Se ha expuesto la precedencia de los operadores.
- Se han ilustrado con ejemplos las expresiones aritméticas.

3. Salida estándar en lenguaje C:

- La salida estándar es la pantalla.
- En lenguaje C se utiliza habitualmente la función `printf` para mostrar información por pantalla.
- Con modificadores y secuencias de escape es posible formatear la salida.

4. Entrada estándar en lenguaje C:

- La entrada estándar es el teclado.
- En lenguaje C se utiliza habitualmente la función `scanf` para interpretar la información introducida por el teclado.
- Para interpretar la información introducida se utilizan una serie de modificadores y una lista de variables donde guardar dicha información.

5. Cálculo de precios según categorías:

- Mediante un ejemplo práctico se ha mostrado cómo implementar código en lenguaje C que nos permita realizar sencillas operaciones numéricas y mostrar información formateada por pantalla.

• Tipos de datos numéricos más habituales:

int float double

- Mostrar información formateada en pantalla:

printf

6. Ficheros en lenguaje C:

- Un archivo o fichero es un conjunto de datos guardado en un dispositivo de almacenamiento.

- Se suceden cuatro fases cuando se trabaja con ficheros en lenguaje C:

- Definir una variable de tipo fichero (FILE *).

- Abrir el fichero (fopen).

- Operar con el fichero (fprintf, fscanf...)

- Cerrar el fichero (fclose).

7. Procedimiento habitual de trabajo con ficheros en lenguaje C:

- Un archivo o fichero es un conjunto de datos guardado en un dispositivo de almacenamiento.

- Se suceden cuatro fases cuando se trabaja con ficheros en lenguaje C:

- Definir una variable de tipo fichero (FILE *).

- Abrir el fichero (fopen).

- Operar con fichero (fprintf, fscanf, ...)

- Cerrar el fichero (fclose).

8. Guardar / en fichero datos de temperaturas:

- Para escribir datos en un fichero de texto sobrescribiendo dicho fichero, usaremos el modo "w" en la función fopen.
- Para escribir datos en un fichero de texto sobrescribiendo dicho fichero, usaremos el modo "a" en la función fopen.

9. Operadores relacionales en lenguaje C:

- Se han introducido los operadores relacionales en C.
- Se ha indicado cómo se evalúan las expresiones relacionales, así como la precedencia de los operadores en la misma, ilustrando todo con ejemplos.

10. Operadores lógicos en lenguaje C:

- Se han introducido los operadores lógicos en C.
- Se ha indicado cómo se evalúan estos operadores según el valor de sus operandos.
- Se ha expuesto la precedencia de los operadores en la misma, ilustrando todo con ejemplos.

11. Sentencia 'if... else' en lenguaje C:

- Es posible variar el flujo de un programa mediante if/else.
- Se ha enunciado cómo implementar sentencias if/else en lenguaje C.
- Se ha ilustrado mediante ejemplos el uso de las sentencias if/else.

12. Cálculo precio billete:

- Mediante un ejemplo práctico se ha mostrado cómo implementar código en lenguaje C que nos permita realizar sencillas operaciones en función de una condición dada.
- Entrada / salida estándar:
 - printf ("Precio del billete:"); - scanf ("%f", &precio);
- Condicionales sencillos en C:
 - if (condición) instrucción a ejecutar si condición es cierta
 - else instrucción a ejecutar si condición es falsa

13. Máximo común divisor y mínimo común múltiplo en lenguaje C:

- Se ha explicado cómo abordar la determinación del mcd y del mcm en lenguaje C.

- Se han indicado dos posibilidades para el uso de parámetros por referencia para la devolución de más de un valor en una función.

14. Resolución de ecuaciones de segundo grado en C:

- Se ha explicado cómo tratar mediante condicionales un problema en el que el cálculo de la solución depende de los datos de entrada.

15. Sentencia switch en lenguaje C:

- Es posible ejecutar distinto código en función del valor de una expresión entera.

- Se ha enunciado cómo utilizar la sentencia switch en lenguaje C.

- Se ha ilustrado mediante ejemplos el uso de las sentencias switch.

16. Errores más comunes en sentencias condicionales en lenguaje C:

- Se han enunciado los errores más frecuentes en el uso de las sentencias condicionales.

- Se ha indicado cómo evitar estos errores.

17. Comprobar si el carácter es una vocal:

- Mediante un ejemplo práctico se ha mostrado cómo implementar código en lenguaje C que nos permita comprobar si un carácter dado es una vocal.

- El tipo de datos char se interpreta como un tipo de datos entero, considerando la correspondencia entre cada carácter y su código ASCII (0-255) correspondiente.

18. Cálculo del número de días entre dos fechas sobre ficheros:
• Se ha explicado cómo implementar código en C que nos permita leer y escribir de forma sencilla en ficheros de texto.
• Se ha mostrado una sencilla estrategia para abordar este tipo de problema, dividiendo la solución entre la lectura de datos, los cálculos y la escritura de datos.

19. Cálculo de la letra NIF en lenguaje C:
• Se ha explicado cómo abordar en lenguaje C la determinación de la letra del NIF de un número de DNI dado.

• Se han indicado diferentes mejoras que hacen que la solución sea más corta y eficiente.

20. Implementación de bucle en C con while:

• La sentencia while permite implementar bucles de forma que una sentencia se repite mientras una condición es cierta.
• La condición se comprueba antes de cada iteración.

21. Implementación de bucles en C con do... while:

• La sentencia do...while permite implementar bucles de forma que una sentencia se repite mientras una condición es cierta.
• La condición se comprueba después de cada iteración.

22. Sumatorio de un número indeterminado de valores:

• Mediante un ejemplo práctico se ha mostrado cómo implementar código en lenguaje C que nos permita resolver un problema en el que hay que repetir un código un número indeterminado de veces.

• Una de las formas más habituales de implementar bucles en C es mediante la instrucción while
- while (condición) cuerpo del bucle

23. Cálculo de cantidades a partir de datos guardados en un fichero:

- Se ha presentado, mediante un ejemplo práctico, el recorrido de datos numéricos almacenados en un fichero.

24. Cálculo del máximo y mínimo de los valores guardados en un fichero:

- Se ha presentado, mediante un ejemplo práctico, cómo calcular los valores máximo y mínimo a partir de los datos numéricos almacenados en un fichero.

25. Cálculo de precio de entrada:

- Mediante un ejemplo práctico se ha mostrado cómo implementar código en lenguaje C que hace uso de operadores lógicos para resolver un problema dado.

- Los operadores lógicos de C son:

- Operador conjunción (y/and) & &

- Operador disyunción (o/or) ||

- Operador negación (no/not) !

26. Implementación de bucles en C con for;

- La sentencia for permite implementar bucles.

- Es especialmente adecuada para bucles en los que se conoce a priori el número de iteraciones a realizar.

27. Cuadrados y cubos de números naturales:

- Mediante un ejemplo práctico se ha mostrado cómo implementar código en lenguaje C que nos permita calcular los cuadrados y los cubos de una relación de números naturales.

- Los resultados se han indicado por pantalla (printf).

- Los resultados se han almacenado en un fichero (fprintf).

28. Programa para el cálculo de la serie de Fibonacci:

- Mediante un ejemplo práctico se ha mostrado cómo calcular términos de la serie de Fibonacci.
- Para pedir un valor al usuario y asegurarse que dicho valor cumpla una condición determinada se suele utilizar un bucle do... while.

29. Vectores y Ficheros: servicio e-Licopter:

- Mediante un ejemplo práctico se ha mostrado cómo utilizar Ficheros en lenguaje C:

- Lectura completa de un fichero de tamaño desconocido a priori.

- Escritura de un fichero.

- Se ha ilustrado el uso de vectores en lenguaje C para la resolución de problemas concretos.

30. Ficheros en lenguaje C: ejemplo de fábrica de nanobots:

- Mediante un ejemplo práctico se ha mostrado cómo utilizar Ficheros en lenguaje C, para lectura completa de un fichero de tamaño desconocido a priori.

- Se ha ilustrado el uso de sentencias condicionales y de repetición para tratamiento de datos.

31. Ficheros en lenguaje C: ejemplo de biblioteca genómica:

- Mediante un ejemplo práctico se ha mostrado cómo utilizar Ficheros en lenguaje C, para lectura completa de un Fichero de tamaño desconocido a priori.

- Se ha ilustrado el uso de sentencias condicionales y de repetición para tratamiento de datos.

32. Evaluación de números primos en lenguaje C:

- Se ha explicado cómo abordar la determinación de números primos en lenguaje C.

- Se ha justificado cómo mejorar la eficiencia a la hora de buscar primos, reduciendo el número de divisores a comprobar.

33. Medir de valores enteros mediante funciones:

• Mediante un ejemplo práctico se ha mostrado cómo calcular la medida de una serie de valores enteros, utilizando una función para ello. Se han presentado dos alternativas en cuanto a la fuente de los datos:

- A partir de tres valores enteros guardados en tres variables diferentes.

- A partir de N valores enteros guardados en un vector.

• Una función es una parte del programa que realiza una tarea específica, y puede ser reutilizada. En una función que distinguir la cabecera y el cuerpo:

$\langle \text{retorno} \rangle$ nombre (tipo1 p1, ...) { cabecera }
 $\langle \text{cuerpo de la función} \rangle$ { cuerpo }

34. Trazas de llamadas a funciones:

• La traza de un código dado es la simulación manual de cómo tendría lugar su ejecución, viendo las repercusiones de la misma.

• Las trazas se utilizan principalmente con dos fines:

- Para identificar posibles errores existentes en el código (especialmente errores semánticos).

- Dado un código para el que se desconoce su cometido, las trazas permiten averiguar el resultado de la ejecución del mismo, facilitando de esta forma el saber cuál es el cometido del código.

35. Cálculo del producto vectorial en C:

• Al utilizarse los vectores como parámetros, pueden utilizarse tanto para consultar los valores de sus elementos, como para actualizar dichos valores.

• La lectura de un vector debe realizarse elemento a elemento.

36. Programa para la Gestión de Asientos de un autobús:

- En muchos casos, es aconsejable llevar a cabo la gestión de un menú de opciones mediante una función para el menú en sí y una función para cada opción.

- Cuando una función tiene vectores como parámetros, éstos son por referencia de manera que en la función se puede:

- Consultar los valores de los elementos del vector.

- Cambiar los valores de los elementos del vector.

37. Totales y media de los elementos de un vector numérico:

- Se ha presentado mediante un ejercicio cómo calcular la suma total de los elementos de un vector numérico y la media de dicho vector.

38. Normalización de los datos de un vector:

- Se ha presentado la implementación de una función para la normalización de un conjunto de datos guardados en un vector.

- Mediante dicha función se ha ilustrado el uso de vectores como argumentos de funciones, así como el manejo de vectores para realizar operaciones con sus elementos.

39. Búsqueda de máximo y mínimo en un vector:

- Se ha presentado la búsqueda de máximo y mínimo en vectores, ilustrándola con ejercicios prácticos resueltos en C.

40. Búsqueda secuencial en vectores:

- Se ha presentado la búsqueda secuencial en vectores, ilustrándola con un ejercicio práctico resuelto en C.

41. Vectores dinámicos en lenguaje C:

- Los vectores dinámicos permiten trabajar con un conjunto de datos homogéneos, pudiéndose establecer el tamaño de dicho conjunto en tiempo de ejecución.

92. Cargar datos en un vector dinámico a partir de un fichero con una cantidad indefinida de valores.

- Mediante un ejemplo práctico se ha mostrado cómo cargar valores desde un fichero, sin conocer a priori la cantidad de valores que hay en el mismo.
- Se ha mostrado también cómo recorrer más de una vez un mismo fichero.

• Se ha ilustrado cómo trabajar con vectores dinámicos:

- Declaración del vector.
- Operaciones con los elementos del vector.
- Reserva de memoria.
- Liberar memoria.

43. Ordenación en C mediante el método de la burbuja:

- El método de ordenación de la burbuja permite ordenar vectores:
 - En sentido ascendente.
 - En sentido descendente.

44. Ordenación en C de datos de vehículos en ficheros:

- Para cargar una serie de valores desde un fichero puede ser recomendable el uso de vectores.
- El método de ordenación de la burbuja permite ordenar vectores con más de un criterio.
- Cuando varios vectores contienen información relacionada por la posición, los intercambios entre elementos deben hacerse en todos los vectores.

45. Ordenación en C de vectores de temperatura y lluvia:

- El método de ordenación de la burbuja permite ordenar vectores con más de un criterio.
- Cuando varios vectores contienen información relacionada por la posición, los intercambios entre elementos deben hacerse en todos los vectores.

46. Tratamiento de cadenas en entrada y salida estándar en C:

- Para mostrar cadenas por pantalla generalmente se utilizan las funciones:

- printf. Permite añadir un texto a la cadena.
- puts. Añade automáticamente un salto de línea tras la cadena.
- Para leer cadenas de teclado habitualmente se usan estas funciones:
- scanf. Lectura de la primera palabra.
- gets. Lectura de toda la cadena (hasta el salto de línea).

47. Funciones más habituales de la librería string.h:

- Se han presentado las funciones de uso más habitual para el manejo de cadenas de la librería string.h, ilustrando su uso con un ejemplo.

48. Cálculo del total de letras de una cadena:

- Se ha presentado, mediante un ejemplo práctico, cómo recorrer una cadena, para interpretar cuáles de sus características corresponden a letras.

49. Tratamiento de cadenas en ficheros con C:

- Para leer cadenas de un fichero generalmente se utilizan las funciones:

- fscanf. Lectura "palabra" o "palabra"
- fgets. Lectura "línea" o "línea"

- Para escribir cadenas en un fichero generalmente se utilizan las funciones:

- fprintf. Permite contextualizar la cadena.
- fputs. No permite contextualizar la cadena.

50. Obtención de una subcadena a partir de una cadena:

- Se ha presentado, mediante un ejemplo práctico, cómo procesar una cadena para obtener una subcadena de la misma.

51. Sustituir un carácter de una cadena por otros caracteres:

- Mediante un ejemplo práctico se ha mostrado cómo trabajar con cadenas carácter a carácter, obteniendo nuevas cadenas a partir de cadenas existentes, incluso cuando las nuevas cadenas tengan un tamaño diferente al de las cadenas originales.
- Recordar el uso de strlen para obtener el tamaño de una cadena.
- Recordar que el \0 determina dónde finaliza una cadena.

52. Ocurrencias de una palabra en un fichero:

- Se ha presentado, mediante un ejemplo práctico, cómo recorrer un fichero de texto palabra a palabra, de manera que se contabilice en total de ocurrencias de una palabra dada.

53. Recorrido de matrices en lenguaje C:

- Se ha indicado cómo realizar el recorrido por los elementos de una matriz, ilustrando todo con un ejercicio práctico resuelto en C.

54. Suma de matrices en lenguaje C:

- Se ha explicado cómo desarrollar código en lenguaje C en el que se realicen operaciones con matrices numéricas, a través de un ejercicio de suma de matrices.

55. Matrices de caracteres en C:

- Se ha explicado cómo trabajar las matrices de caracteres como vectores de cadenas, de forma que sólo se realiza un bucle (en lugar de dos bucles anidados como suele ocurrir con otro tipo de matrices).

56. Invertir una imagen pgm:

- Mediante un ejemplo práctico se ha mostrado cómo implementar código en lenguaje C que nos permita:
- Cargar imágenes guardadas en matrices numéricas.
- Guardar imágenes en un fichero de texto a partir de matrices numéricas.

57. Caracterización de alumnos mediante estructuras en lenguaje C:
• Se ha presentado, mediante un ejemplo práctico, cómo declarar estructuras en C y cómo implementar sencillas funciones que las reciban como parámetro.

58. Estructuras y funciones: sets de juego:

• Mediante un ejemplo práctico se ha mostrado cómo implementar estructuras que se ajusten a unas necesidades de almacenamiento determinadas.

• Se ha ilustrado igualmente el desarrollo de funciones que reciban estructuras y vectores de estructuras.

59. Estructuras y funciones: motores de clientes:

• Mediante un ejemplo práctico se ha mostrado cómo implementar estructuras que se ajusten a unas necesidades de almacenamiento determinadas.

• Se ha ilustrado igualmente el desarrollo de funciones que reciban vectores de estructuras, procesándolos para conseguir lo pedido.

• Se ha visto cómo comparar cadenas de caracteres (strcmp).

• Se ha indicado cómo llamar a funciones que reciban vectores de estructuras.

60. Estructuras y funciones en lenguaje C: ejemplo de robot eléctricos:

• Mediante un ejemplo práctico se ha mostrado cómo implementar estructuras que se ajusten a unas necesidades de almacenamiento determinadas.

• Se ha ilustrado igualmente el desarrollo de funciones que reciban estructuras o vectores de estructuras, procesándolos para conseguir lo pedido.

• Se ha visto cómo comparar cadenas de caracteres (strcmp).

61. Uso de estructuras en C:

• Se ha presentado, mediante un ejemplo práctico, cómo declarar estructuras en C y cómo implementar sencillas funciones que las reciban como parámetro.